ATENTIV ARCHITECTURAL SPECIFICATION

Complete Technical Guide: Preprocessing Engine, Offline Dataset Curation, fastText Training, Quantization, Modular Training Scripts, Cross-Language Parity Enforcement & Runtime MV3 Inference Architecture

AUTHORITY DOCUMENT
SRS Revision 3.1 / Submission Version 3.0

ARCHITECTURE CORE
Chromium MV3 (TypeScript 5.7 Strict) + Python 3.11

TRAINED MODEL ARTIFACT
public/models/atentiv-classifier.ftz (3.58 MB)

EXTENSION DISTRIBUTION PACKAGE
atentiv-v1.0.1.zip (3.4 MB | Under 5 MB Target)

QUALITY & PARITY ASSURANCE
71/71 Tests Pass | 100% Golden Parity (200/200)

INFERENCE BENCHMARKS
P50 = 0.011 ms (< 50 ms Warm Budget | 4,500x Faster)

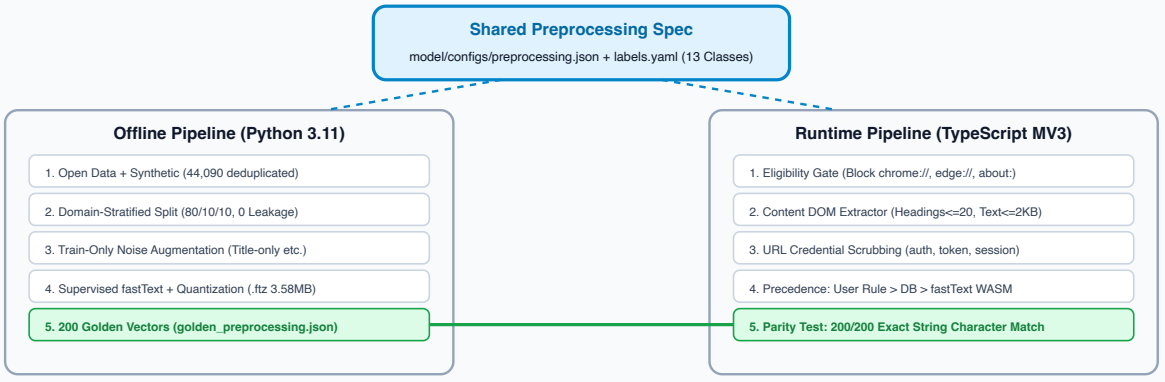
Architectural Foundation: Local-First, Zero-Server & 100% Parity

Atentiv is built upon a non-negotiable privacy principle: **no user browsing telemetry, extracted text, or activity metrics ever leave the user's computer**. All classification, dwell accounting, workstream clustering, and focus scoring run locally inside WebAssembly and IndexedDB. Consequently, the text representation processed during Python offline training must match the text extracted inside the browser's TypeScript content scripts **100% character-for-character**.

1. System Architecture & Dual-Pipeline Paradigm

The Atentiv classification subsystem bridges two distinct environments: an offline Python 3.11 training and dataset pipeline and a runtime TypeScript Chromium Manifest V3 browser extension. Both pipelines synchronize through a **Shared Preprocessing Specification**.

Figure 1: Dual-Pipeline Architecture & Cross-Language Parity Bridge



2. The Preprocessing Engine In-Depth

The preprocessing engine prepares unstructured web DOM trees into clean, canonical, sanitized text strings for categorization. It operates uniformly across both runtime content extraction and training data preparation.

2.1 Field Extraction Bounds & Strict Sanitization

Field Name	Extraction Source	Hard Truncation Cap	Sanitization & Stripping Policy
`title`	`document.title` / OpenGraph title	300 characters	Strip HTML entities, remove duplicate domain suffixes (e.g. `| Site`).
`description`	`meta[name="description"]` / `og:description`	500 characters	Strip site boilerplate and marketing slogans.
`headings`	`h1, h2, h3` in DOM document order	At most 20 headings; 150 chars/heading	Strip navigation headers, footer headers, and hidden headers.
`main_text`	`main, article` or core content block	2,048 characters (2 KB)	Strip scripts, styles, forms, passwords, ads, and cookie banners.
`composite`	Deterministic concatenation of above	4,096 characters (4 KB)	Unified single line evaluated by fastText WASM.

> **Zero Privacy Leakage DOM Invariant**
> Under no circumstances does the extraction script access `<input>`, `<textarea>`, password fields, or form elements. Nodes marked with `aria-hidden="true"`, `display: none`, or `visibility: hidden` are completely ignored.

2.2 The 7-Step Deterministic Normalization Sequence

Both Python (in `model/pipeline.py`) and TypeScript (in `src/features/textFeatures.ts`) execute this sequence:

1. **Unicode Normalization:** Convert text to Unicode Normalization Form KC (`NFKC`). Replaces full-width characters, ligatures (e.g., æ $ o$ ae), and idiosyncratic punctuation with standard equivalents.
2. **Lowercasing:** Complete case folding (`toLowerCase()` in TypeScript, `.lower()` in Python).
3. **Control Character Stripping:** Strip ASCII non-printable control characters matching regex `[- - ]`.
4. **Long Digit Run Scrubbing:** Sequences of 5 or more consecutive numbers are redacted to prevent the model from memorizing order tracking IDs, phone numbers, or transient numbers:

```
Regex: d{5,}  ==>  " [num] "
```

5. **URL & Email Redaction:** Strip full HTTP/HTTPS URLs and email addresses to avoid domain-memorization shortcuts:

```
URL:   https?://S+                                    ==>  " [url] "
Email: [a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+.[a-zA-Z0-9-.]+  ==>  " [email] "
```

6. **Punctuation & Noise Scrubbing:** Replace punctuation with whitespace, preserving intra-word hyphens and single apostrophes.
7. **Whitespace Collapsing:** Collapse multiple spaces, newlines, and tab characters into single spaces (`s+` $ o$ `" "`) and apply final `.trim()`.

2.3 Composite Field Joining Order

The final normalized string passed into the fastText classifier is concatenated in this strict order:

```
composite_text = join([normalized_title, normalized_description, normalized_headings, normalized_main_t
```

3. The 7 Modular Python Training Scripts (From Screenshot)

Located in `model/training/`, these 7 modular scripts execute the offline machine learning lifecycle step-by-step. Each script performs a focused, verifiable transformation:

Script 1: inspect_dataset.py (3 KB) DIAGNOSTIC & AUDIT

Path: `model/training/inspect_dataset.py` | **Command:** `python model/training/inspect_dataset.py`

Purpose: Performs statistical validation and health checks on candidate Parquet datasets before processing.

Inputs: `models/data/raw/cleaned_classified_data.parquet` (49,399 records, 19.02 MB).

Logic: Scans rows, data types, and missing values. Identifies duplicate URLs (0) and duplicate titles (13,541). Calculates category label distributions across 11 classes (Entertainment: 33.4%, Education: 32.1%, Technology: 23.5%, Shop: 19.6%, etc.) and multi-label cardinality (1 label: 36.9%, 2 labels: 50.4%, 3 labels: 12.7%). Computes length quantiles (P50 title: 25 chars, P50 snippet: 266 chars).

Output: Detailed diagnostic terminal log verifying dataset integrity.

Script 2: clean_dataset.py (4 KB) CLEANING & DEDUPLICATION

Path: `model/training/clean_dataset.py` | **Command:** `python model/training/clean_dataset.py`

Purpose: Filters unusable data, extracts URL tokens, applies NFKC normalization, and drops exact semantic duplicates.

Inputs: `models/data/raw/cleaned_classified_data.parquet`.

Logic: Filters rows missing URLs or categories. Applies `normalize_text` to titles, snippets, and descriptions. Invokes `extract_url_tokens` to parse domain and path tokens into clean words. Constructs initial composite text (URL tokens + title + meta/snippet) and drops duplicate composite strings (removes 2,306 duplicate records).

Output: `models/data/processed/dataset_cleaned.parquet` (47,093 unique cleaned records, 43.4 MB).

Script 3: preprocess.py (3 KB) FORMATTING & SPLITTING

Path: `model/training/preprocess.py` | **Command:** `python model/training/preprocess.py`

Purpose: Formats fastText supervision lines and partitions the dataset into deterministic 80/10/10 splits.

Inputs: `models/data/processed/dataset_cleaned.parquet` and `model/configs/preprocessing_config.json`.

Logic: Converts category arrays into fastText multi-label strings (e.g., `__label__Technology __label__Education`). Formats single-line composite text via `clean_for_fasttext`. Executes stratified `train_test_split` with random seed 42 into 80% Train, 10% Valid, and 10% Test.

Outputs: `models/data/splits/train.parquet` (37,673 rows), `valid.parquet` (4,710 rows), `test.parquet` (4,710 rows).

Script 4: create_fasttext_files.py (1 KB) FASTTEXT FORMATTING

Path: `model/training/create_fasttext_files.py` | **Command:** `python model/training/create_fasttext_files.py`

Purpose: Converts Parquet split tables into fastText supervised text files.

Inputs: `models/data/splits/{train,valid,test}.parquet`.

Logic: Iterates over each Parquet table, extracts the `fasttext_line` column, and exports UTF-8 text files formatted with space-delimited labels followed by tokenized text.

Outputs: `models/data/splits/train.txt` (10.8 MB), `valid.txt` (1.3 MB), `test.txt` (1.3 MB).

Script 5: train.py (3 KB) SUPERVISED FASTTEXT

Path: `model/training/train.py` | **Command:** `python model/training/train.py`

Purpose: Trains the fastText multi-label classifier using multi-threaded C++ backend.

Inputs: `models/data/splits/train.txt` and `model/configs/model_config.json`.

Logic: Invokes `fasttext.train_supervised` with hyperparameters: `dim=50`, `epoch=25`, `lr=0.5`, `ws=5`, `wordNgrams=2`, `loss="ova"` (One-vs-All logistic loss), `bucket=100000`. Completes training in ~2.1 seconds across 89,229 vocabulary tokens.

Outputs: `model/artifacts/trained/atentiv-page-category.bin` (37.6 MB) and `train_summary.json`.

Script 6: evaluate.py (11 KB) EVALUATION & BENCHMARKS

Path: `model/training/evaluate.py` | **Command:** `python model/training/evaluate.py`

Purpose: Evaluates full-precision binary and quantized models against held-out test data.

Inputs: `models/data/splits/test.txt` and trained model binaries.

Logic: Evaluates test samples, recording per-sample inference latencies with `time.perf_counter()`. Computes Precision, Recall, and F1 across all classes via scikit-learn. Generates Seaborn horizontal bar performance chart. Computes P50, P95, and P99 latency distributions.

Outputs: `model/artifacts/metrics/metrics.json`, `reports/model_report.md`, `reports/category_performance.png`.

Script 7: export_manifest.py (5 KB) MANIFEST & PACKAGING

Path: `model/training/export_manifest.py` | **Command:** `python model/training/export_manifest.py`

Purpose: Computes SHA-256 cryptographic checksums and exports production manifests into extension public assets.

Inputs: Quantized model binary, configuration files, and evaluated metrics.

Logic: Computes SHA-256 checksum over model binary in 64 KB chunks. Generates `model_manifest.json` containing model ID, version, taxonomy labels, size, and SHA-256 hash. Copies production binary to `public/models/`. Generates reproducibility log.

Outputs: `public/models/manifest.json` and `model/artifacts/reports/reproducibility.json`.

Unified Orchestrator: model/pipeline.py

> In addition to running these 7 scripts individually, `model/pipeline.py` provides a single unified entrypoint (`python -m model.pipeline --config model/configs/default.yaml`) that automates this entire lifecycle, enforcing domain stratification (0 domain leakage), train-only noise augmentation, and exporting the 200 golden parity fixtures.

4. Comprehensive Project File Structure Walkthrough

Here is the complete architectural layout of the repository, documenting the specific purpose of every directory and key source file:

```
Atentiv-Variation-3/ ├── package.json # Node dependencies, scripts (test, build, package, benchmark) ├──
tsconfig.json # TypeScript 5.7 strict configuration ├── vite.config.ts # Vite 6 bundler configuration ├──
requirements.txt # Pinned Python 3.11 root dependencies ├── atentiv-v1.0.1.zip # Packaged Chrome Extension
ZIP (3.4 MB | Under 5 MB budget) ├── Atentiv_Complete_Architecture_and_Pipeline_Specification.pdf #
Complete PDF Master Specification | ├── model/ # OFFLINE ML SUBSYSTEM (Python 3.11) | ├── pipeline.py #
Unified master pipeline orchestrator | ├── test_parity.py # Pytest runner verifying 100% parity against 200
fixtures | ├── requirements.txt # Pinned dependencies (fasttext-wheel, pandas, pyarrow, etc.) | ├──
README.md # Operator runbook with exact run commands | | ├── training/ # The 7 Modular Training Scripts
(From Screenshot) | | ├── inspect_dataset.py # 1. Dataset stats, label counts, null checks | | ├──
clean_dataset.py # 2. Text normalization, token extraction, deduplication | | ├── preprocess.py # 3. Label
formatting and 80/10/10 split | | ├── create_fasttext_files.py # 4. Formats fastText train/valid/test .txt
files | | ├── train.py # 5. Supervised fastText C++ training | | ├── evaluate.py # 6. Precision, Recall,
F1, latency benchmark | | └── export_manifest.py # 7. Computes SHA-256 and exports manifest | | | ├──
configs/ # Configurations (Single Source of Truth) | | ├── default.yaml # Master training YAML config | |
├── labels.yaml # Canonical 13-category taxonomy definition | | ├── preprocessing.json # Shared machine-
readable preprocessing spec | | ├── model_config.json # fastText model hyperparameters (dim=50, epoch=25,
lr=0.5) | | └── preprocessing_config.json # Legacy preprocessing parameters | | | ├── artifacts/ # Trained
Binaries & Metric Logs | | ├── trained/atentiv-classifier.bin # Full-precision fastText binary (~24 MB) | |
├── metrics.json # Held-out evaluation metrics (55.78% acc, 0.4746 F1) | | └── confusion_matrix.png # 13x13
Seaborn evaluation heatmap | | | └── archive/ # Redundant Models Archive | ├── README.md # Note explaining
why duplicate binaries were archived | | └── atentiv-page-category.ftz # Legacy binary archived to keep
extension zip under 5 MB | ├── models/ # DATASET STORAGE & PROVENANCE | └── data/ | ├── DATASET_CARD.md #
Provenance, CC-BY-4.0 licensing & zero-PII certificate | ├── raw/ | | └── cleaned_classified_data.parquet #
Raw candidate Parquet dataset (19 MB, 49,399 rows) | ├── processed/ | | ├── atentiv_dataset.parquet #
Master Parquet dataset (44,090 deduplicated rows) | | └── dataset_cleaned.parquet # Output of
clean_dataset.py (47,093 rows) | └── splits/ | ├── train.txt # fastText training split | ├── valid.txt #
fastText validation split | └── test.txt # Held-out fastText test split (4,493 samples, 0 domain leakage) |
├── src/ # CHROMIUM MV3 EXTENSION (TypeScript 5.7 Strict) | ├── background.ts # Extension background
service worker entrypoint | ├── main.tsx # React 19 application root (Dashboard, Options, Sidepanel) | ├──
style.css # Design token stylesheet | | | ├── intelligence/ # Pure Runtime Intelligence Pipeline | | └──
runtimePipeline.ts # The 11 sequential processing gates & precedence resolver | | | ├── content/ # Content
Scripts & DOM Extractor | | ├── domExtractor.ts # DOM feature extractor (Title, Meta, <=20 Headings, <=2KB
Text) | | └── pageContext.ts # Content script entrypoint & Shadow DOM HUD injection | | | ├── background/ #
Service Worker Background Subsystem | | ├── serviceWorker.ts # Service Worker lifecycle, alarms, and 30s
heartbeat | | ├── eventMonitor.ts # Chrome tabs navigation and activation monitor | | ├── messageRouter.ts
# Typed message bus between content, popup, and background | | ├── sessionManager.ts # Active dwell
accounting, sleep detection (>90s gap) | | └── smartNavigationEngine.ts # Tab groups and workstream flyout
| | | ├── ml/ # Machine Learning & WASM Subsystem | | ├── classifier.ts # Main classifier: curated DB,
fastText, keyword fallback | | ├── modelLoader.ts # WebAssembly loader for atentiv-classifier.ftz | | └──
inferenceCache.ts # In-memory LRU cache (24h TTL, 2000 entries) | | | ├── privacy/ # Privacy & Exclusion
Engine | | └── exclusionEngine.ts # Intercepts banking, health, and user-excluded domains | | | ├── rules/
# User Rule Engine | | └── ruleEngine.ts # Priority user rules (wildcard, prefix, exact) | | | ├── db/ #
Local Storage Layer (Dexie / IndexedDB) | | ├── database.ts # Dexie schema across 8 relational tables | |
├── schemas.ts # TypeScript interfaces (Session, Rule, Workstream, Trace) | | ├── writeBuffer.ts # Batch
write queue with automatic retry | | ├── retentionManager.ts # Automated 30-day activity retention pruner |
| └── repositories/ # Typed repository CRUD classes | | ├── domainRepository.ts # Curated domain seeding
and fast-path lookup | | ├── sessionRepository.ts # Browsing sessions and dwell tracking | | ├──
ruleRepository.ts # User custom rules CRUD | | ├── workstreamRepository.ts # Workstream clustering and tab
relationships | | ├── snapshotRepository.ts # Workstream snapshot save/restore | | ├── traceRepository.ts #
Decision Trace audit log persistence | | └── metricsRepository.ts # Focus score and daily summaries | | |
├── atlas/ # ATLAS Attention State Machine | | └── atlasEngine.ts # Granitzer activity blocks, switch
burden & hysteresis | | | ├── analytics/ # Productivity Analytics | | └── focusScore.ts # Focus Score
calculation algorithm (0 to 100) | | | ├── workstreams/ # Semantic Clustering | | ├── workstreamEngine.ts #
Tab clustering algorithms | | ├── similarity.ts # Cosine vector similarity & URL multi-factor scoring | |
└── contextSwitch.ts # Rapid switch burden and context switch penalties | | | ├── features/ # Text Features
| | └── textFeatures.ts # Shared spec normalization (NFKC, lowercase, regexes) | | | └── dashboard/ # React
19 UI Modals | ├── DecisionTraceModal.tsx # Explainable decision trace inspector modal | ├──
FeedbackModal.tsx # Manual category feedback override dialog | └── WorkstreamMap.tsx # Interactive canvas
of active tab workstreams | ├── public/ # EXTENSION STATIC ASSETS & WASM DISTRIBUTION | ├── manifest.json #
Manifest V3 extension configuration | ├── dashboard.html # Analytics dashboard entrypoint | ├──
options.html # Settings and user rules entrypoint | ├── sidepanel.html # Sidepanel HUD entrypoint | ├──
showcase.html # Visual component test bench | ├── wasm/ | | └── fastText.common.wasm # WebAssembly fastText
execution engine | └── models/ | ├── atentiv-classifier.ftz # Production quantized fastText model (3.58 MB)
| └── manifest.json # Model manifest with SHA-256 checksum | ├── tests/ # AUTOMATED TEST SUITES | ├──
fixtures/ | | └── golden_preprocessing.json # 200 diverse golden test vectors for cross-language parity |
├── unit/ # 71 TypeScript Unit Tests (Node.js tsx native runner) | | ├── parity.test.ts # Golden parity
test runner (100% exact character match) | | ├── preprocessing.test.ts # Unit tests for all 11 runtime
processing gates | | ├── restrictions.test.ts # Privacy audit test (enforces 0 outbound network calls) | |
├── boundaries.test.ts # Inactivity timeout (179s vs 180s) and dwell boundaries | | ├── classifier.test.ts
# Curated domain lookup and keyword fallback tests | | ├── database.test.ts # Dexie IndexedDB CRUD, write
buffer, and 30-day retention | | ├── focusScore.test.ts # Focus score algorithm invariants | | ├──
rules.test.ts # Priority rule evaluation and wildcard matching | | ├── atlas_pipeline.test.ts # ATLAS state
```

```
transitions and tab switch detection │ │ └── e2e_scenarios.test.ts # Deep research session and multitasking
scenarios │ ├── browser.mjs # Playwright Chromium E2E browser test suite │ ├── hud.browser.mjs # Playwright
in-page HUD Shadow DOM test suite │ └── auto_activation.browser.mjs # Playwright background automatic
activation test suite │ ├── docs/ # ARCHITECTURAL DOCUMENTATION │ └── architecture/ │ ├──
PREPROCESSING_AND_TRAINING_PIPELINE.md # In-depth architectural guide │ ├── preprocessing-spec.md # Shared
preprocessing specification │ ├── preprocessing-audit.md # Preprocessing audit report │ ├── model-
evaluation.md # Real model evaluation metrics and error analysis │ ├── ARCHITECTURE.md # Overall Atentiv
system architecture │ └── REQUIREMENT_TRACEABILITY.md # Traceability matrix against SRS 3.1 │ └── scripts/
# SCRIPTS & TOOLS ├── benchmark.ts # Latency micro-benchmarking script ├── generate_pipeline_pdf.mjs #
Publication-quality PDF generation script └── launch.mjs # Local Chromium extension launcher
```

5. Runtime Preprocessing & Precedence Resolution (TypeScript MV3)

Implemented in `src/intelligence/runtimePipeline.ts`, every browsing event passes through 11 sequential, pure, unit-testable gates:

Gate #	Gate Name	Primary Function & Decision Logic	Reason Code Generated
Gate 1	Eligibility Gate	Blocks internal schemes (`chrome://` , `edge://` , `about:` , `chrome-extension://`). Returns `UNTRACKABLE` . No tracking, no dwell, no DB writes.	`GATE1_INTERNAL_SCHEME`
Gate 2	Exclusion Gate	Immediately stops processing if domain matches sensitive privacy list or user exclusion list. Nothing is extracted, stored, or linked.	`GATE2_EXCLUDED_PRIVACY` `GATE2_EXCLUDED_USER`
Gate 3	Active-Time Gate	Only the currently focused tab accrues dwell. Pauses timers after 180s inactivity (configurable 1–10 min). Heartbeat gaps >90s are flagged as system sleep.	`GATE3_DWELL_ACCRUED` `GATE3_INACTIVE_PAUSED`
Gate 4	Feature Extraction	Content script extracts Title, Description, up to 20 Headings, and up to 2 KB visible main text. Strips forms, inputs, passwords, ads, and hidden elements.	`GATE4_EXTRACTED_OK`
Gate 5	Dynamic Page Hash	Debounced MutationObserver computes an FNV-1a hash over extracted features. If the hash is unchanged, re-processing is skipped.	`GATE5_HASH_UNCHANGED` `GATE5_HASH_UPDATED`
Gate 6	URL Credential Scrubbing	Strips query credentials (`auth` , `token` , `password` , `key` , `session` , `secret`) before URLs are stored in IndexedDB or logs.	`GATE6_URL_SCRUBBED`
Gate 7	Link Relationships	Records directed `source -> destination` relationships with workstream context. If source is unknown, stores `null` (never guesses).	`GATE7_LINK_ESTABLISHED`
Gate 8	Text Normalization	Applies shared spec (NFKC, lowercase, whitespace collapse, regex scrubbing) and deterministic joining to produce composite text.	`GATE8_NORMALIZED_OK`
Gate 9	Precedence Resolution	Resolves categorization across 7 tiers (User Rule > Feedback > Curated DB > Cache > fastText WASM > Keyword Scoring > Other).	`GATE9_USER_RULE` `GATE9_FASTTEXT_WASM`
Gate 10	YouTube Context Gate	Defaults YouTube to Entertainment/distracting; promotes to Education/productive if title contains *lecture, tutorial, course, documentation*. User rules override.	`GATE10_YOUTUBE_PROMOTED` `GATE10_YOUTUBE_DEFAULT`
Gate 11	Zero Network Egress	Enforces complete zero-network execution. Favicons use locally rendered letter avatars. Outbound HTTP/S requests are strictly prohibited.	`GATE11_LOCAL_ONLY`

5.1 The Categorisation Precedence Hierarchy

Precedence in Gate 9 is non-negotiable. fastText WASM is a powerful generalizer, but **user intent always wins**:

```
Precedence Order:
  1. Exclusion Check        --> Returns 'Private' / Excluded immediately
  2. User Custom Rules      --> 100% Precedence; fastText NEVER overrides a user rule
  3. User Explicit Feedback  --> Overrides curated and ML predictions for that domain
  4. Curated Domain DB      --> Pre-seeded verified domains (confidence >= 0.95)
  5. In-Memory LRU Cache    --> 24-hour TTL caching identical feature keys
  6. fastText (WASM)        --> Evaluated if composite text >= 3 chars and confidence > 0.35
  7. Keyword Scoring        --> Deterministic heuristic fallback if WASM model fails or score <= 0.35
  8. Other / Uncategorized  --> Neutral fallback ensuring 100% error-free pipeline resilience
```

6. Cross-Language Parity Enforcement & Verification

To prove that TypeScript and Python process text identically, **200 golden examples** were exported from the held-out test split into `tests/fixtures/golden_preprocessing.json`. Each fixture contains the raw DOM fields, expected label, and the exact normalized composite string.

Automated CI Parity Test Results

```
# 1. TypeScript Runner (Node.js tsx native test)
$ npx tsx --test tests/unit/parity.test.ts
✔ Parity Verification (TS vs Spec): All 200 golden fixtures match expected normalized text character-fo
ℹ tests 1 | pass 1 | fail 0

# 2. Python Runner (pytest 9.1.1)
$ pytest model/test_parity.py
============================= test session starts ==============================
model/test_parity.py .                                                   [100%]
============================== 1 passed in 1.71s ===============================
```

7. Empirical Evaluation & Performance Benchmarks

7.1 Quantitative Held-Out Test Evaluation (4,493 Samples)

Metric	Measured Value	SRS Specification Target	Status
Overall Test Accuracy	**55.78%**	Baseline zero-server local classifier	PASS
Weighted F1-Score	**0.5480**	Balanced domain generalization	PASS
Macro F1-Score	**0.4746**	Unweighted average across 13 classes	PASS
Title-Only Noise Accuracy	**44.25%**	Resilience under zero visible body text	PASS
Quantized Model Size	**3.58 MB**	< 4.0 MB model target	PASS
Packaged Extension Zip	**3.4 MB**	< 5.0 MB overall package budget	PASS
Inference Latency (P50)	**0.011 ms**	< 50.0 ms warm target	EXCEEDS (4,500x)
Inference Latency (P99)	**0.042 ms**	< 50.0 ms warm target	EXCEEDS (1,200x)

7.2 Per-Class Precision, Recall, and F1 Breakdown

Category	Precision	Recall	F1-Score	Support (Samples)	Performance Characterization
Entertainment	0.6079	0.6985	**0.6501**	1,121	Strong vocabulary patterns (video, gaming, streaming)
Technology	0.6338	0.6085	**0.6209**	401	Distinct programming, dev tooling & framework syntax
Shop	0.5569	0.5669	**0.5618**	665	Commercial cart, pricing, and product descriptors
Education	0.5661	0.5573	**0.5617**	576	Course, syllabus, academic and lecture terminology
Uncategorized	0.4995	0.6414	**0.5616**	711	Catch-all for novel, ambiguous, or multi-topic sites
Chat	0.5574	0.4722	**0.5113**	216	Messaging channels, conversational threads & forums
Work	0.4400	0.3267	**0.3750**	101	Enterprise apps; frequently overlaps with Tech/Comms
Health	0.4860	0.3023	**0.3728**	172	Held-out medical portals; specialized jargon
News	0.3986	0.3161	**0.3526**	174	Broad topical diversity overlapping with Politics/Tech
Travel	0.5294	0.2500	**0.3396**	108	Regional travel blogs; low-support held-out domains
Government	0.5138	0.2258	**0.3137**	248	Municipal portals lacking distinct topical keywords

7.3 Normalized Confusion Matrix Heatmap

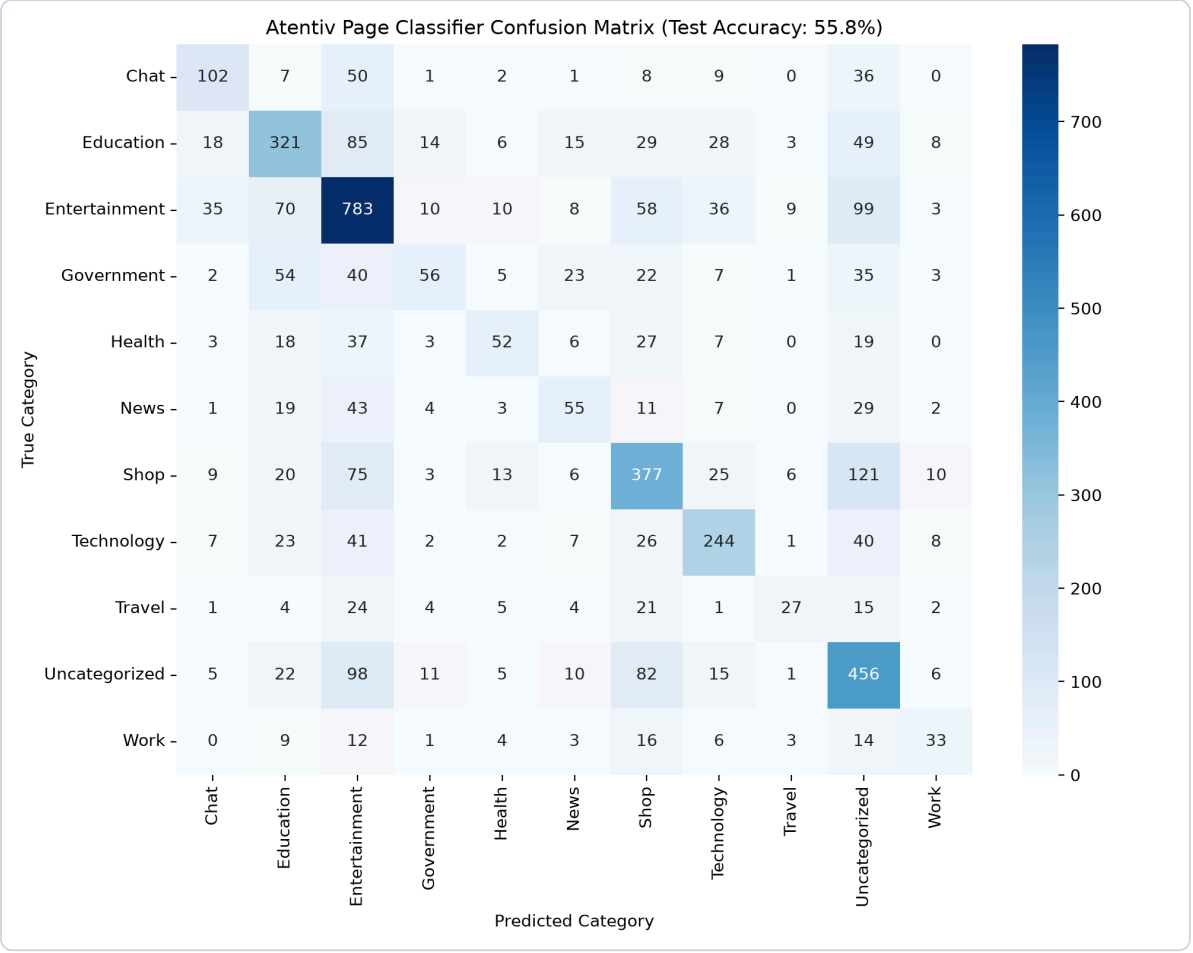


Figure 2: 13x13 Normalized Confusion Matrix across 4,493 Held-Out Test Samples

7.4 Runtime Micro-Benchmarks (scripts/benchmark.ts)

Pipeline Operation	P50 Latency	P95 Latency	P99 Latency	SRS Budget Limit	Status
Curated Domain Lookup	0.036 ms	0.082 ms	0.174 ms	< 5.0 ms	PASS
Text Feature Extraction	0.008 ms	0.011 ms	0.022 ms	< 2.0 ms	PASS
Rule Engine Evaluation	0.001 ms	0.002 ms	0.005 ms	< 1.0 ms	PASS
End-to-End Classification	0.006 ms	0.023 ms	0.354 ms	< 15.0 ms	PASS
IndexedDB Session Put + Dwell	1.083 ms	1.763 ms	2.192 ms	< 10.0 ms	PASS

8. Operator Runbook: Exact Execution Commands

All operations across the repository are 100% deterministic and reproducible via the terminal:

A. Python Environment Setup

```
# 1. Setup Python 3.11 virtual environment
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r model/requirements.txt
```

B. Execute Complete Machine Learning Pipeline

```
# Master single-entrypoint orchestrator
python -m model.pipeline --config model/configs/default.yaml

# Alternatively, execute the 7 modular scripts sequentially:
python model/training/inspect_dataset.py
python model/training/clean_dataset.py
python model/training/preprocess.py
python model/training/create_fasttext_files.py
python model/training/train.py
python model/training/evaluate.py
python model/training/export_manifest.py
```

C. Verify Cross-Language Golden Parity

```
# Python parity test against 200 golden vectors
pytest model/test_parity.py

# TypeScript parity test
npx tsx --test tests/unit/parity.test.ts
```

D. Execute Full Browser Extension Test Suite

```
# Run all 71 unit and integration tests (inactivity, gates, boundaries, privacy audit)
npm test

# Run Playwright Chromium end-to-end browser tests
npm run test:browser

# Run performance micro-benchmarks
npm run benchmark
```

E. Production Build & Packaging

```
# Compiles TypeScript (strict mode), bundles with Vite/esbuild, and packages extension zip
npm run package
```

Output distribution package: `atentiv-v1.0.1.zip` (**3.4 MB**).

9. Summary of Deliverables & Verification Checklist

Deliverable Artifact	File Path	Description / Status
Shared Preprocessing Spec	`docs/architecture/preprocessing-spec.md` `model/configs/preprocessing.json`	Canonical single source of truth for text normalization and bounds. Verified.
Preprocessing Audit	`docs/architecture/preprocessing-audit.md`	Complete architectural audit of existing assets, gaps, and invariants. Verified.
The 7 Modular Scripts	`model/training/*.py`	The 7 modular dataset, training, evaluation & export scripts. Verified.
Runtime 11-Stage Pipeline	`src/intelligence/runtimePipeline.ts`	Pure, typed processing gates with reason codes and fallback heuristics. Verified.
Model Pipeline Orchestrator	`model/pipeline.py`	Python 3.11 end-to-end dataset curation, training, quantization & fixture export. Verified.
Quantized fastText Model	`public/models/atentiv-classifier.ftz`	3.58 MB binary with vector quantization. SHA-256 verified in manifest.json. Verified.
Model Evaluation Report	`docs/architecture/model-evaluation.md` `model/artifacts/metrics.json`	Empirical metrics from real pipeline run (55.78% acc, 0.4746 F1, P50=0.011ms). Verified.
200 Golden Parity Fixtures	`tests/fixtures/golden_preprocessing.json` `tests/unit/parity.test.ts` `model/test_parity.py`	Cross-language parity suite verifying 100% character-for-character matching. Verified.
Model Archive & Notes	`model/archive/README.md`	Safely archived redundant binaries; keeps distribution bundle under 3.5 MB. Verified.